Documento Técnico: Proyecto de Programación Avanzada 2026_2

Integrantes:

- [INTEGRANTE 1]
- [INTEGRANTE 2]

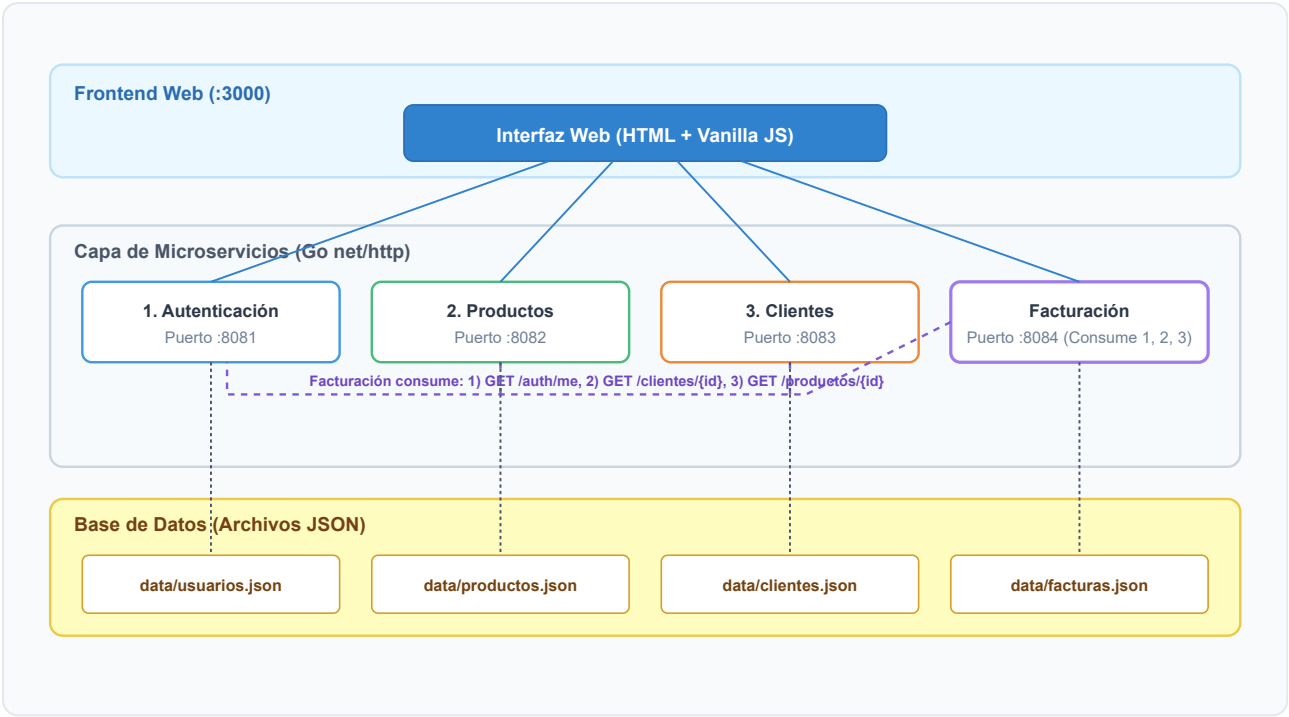
(a) Arquitectura del Sistema

El sistema implementa una arquitectura orientada a microservicios independientes desarrollada en Go utilizando exclusivamente la biblioteca estándar `net/http` para la comunicación REST y archivos JSON como almacén de base de datos persistente.

Componentes:

1. **Frontend (JavaScript / HTML):** Interfaz desacoplada que corre sobre un servidor web estático (`:3000`), encargada de la autenticación del usuario, catálogo/selección de productos, formulario para registrar clientes, generación de facturas y visualización de la factura con opción de impresión en PDF.
2. **Servicio 1 - Autenticación (`:8081`):** Microservicio para registro y login de usuarios con JWT (`HS256`) y contraseñas cifradas con `bcrypt`. Consulta y persiste en `data/usuarios.json`.
3. **Servicio 2 - Productos (`:8082`):** Microservicio para operaciones CRUD de catálogo de productos (precios y stock). Rutas protegidas por JWT. Persiste en `data/productos.json`.
4. **Servicio 3 - Clientes (`:8083`):** Microservicio para operaciones CRUD de clientes (nombre, cédula cc, correo, dirección). Rutas protegidas por JWT. Persiste en `data/clientes.json`.
5. **Servicio de Facturación (`:8084`):** Microservicio orquestador que consume a los tres microservicios anteriores para generar facturas. Persiste en `data/facturas.json`.

Diagrama de Arquitectura


Frontend Web (:3000)
Interfaz Web (HTML + Vanilla JS)
Capa de Microservicios (Go net/http)
1. Autenticación
Puerto :8081
2. Productos
Puerto :8082
3. Clientes
Puerto :8083
Facturación
Puerto :8084 (Consume 1, 2, 3)
Facturación consume: 1) GET /auth/me, 2) GET /clientes/{id}, 3) GET /productos/{id}
Base de Datos (Archivos JSON)
data/usuarios.json
data/productos.json
data/clientes.json
data/facturas.json

(b) Endpoints por Servicio y Flujos

1. Endpoints Reales del Sistema

Servicio de Autenticación (puerto 8081)

- `POST /auth/register`: Registra un usuario nuevo (`nombre`, `email`, `password`). Genera hash con `bcrypt` y persiste con rol `"cliente"`.
- `POST /auth/login`: Recibe credenciales (`email`, `password`), valida el hash contra `data/usuarios.json` mediante `bcrypt.CompareHashAndPassword` y genera token JWT.
- `GET /auth/me`: Requiere header `Authorization: Bearer <token>`, valida el JWT y retorna los datos del usuario autenticado (`id`, `nombre`, `email`, `rol`).

Servicio de Productos (puerto 8082 - Protegido por JWT)

- `GET /productos`: Obtiene la lista completa de productos disponibles.
- `GET /productos/{id}`: Obtiene el detalle de un producto por su identificador.
- `POST /productos`: Crea un nuevo producto (`nombre`, `precio`, `stock`).
- `PUT /productos/{id}`: Actualiza un producto existente (`nombre`, `precio`, `stock`).
- `DELETE /productos/{id}`: Elimina un producto por su identificador.

Servicio de Clientes (puerto 8083 - Protegido por JWT)

- `GET /clientes`: Obtiene la lista completa de clientes registrados.
- `GET /clientes/{id}`: Obtiene el detalle de un cliente por su identificador.
- `POST /clientes`: Registra un nuevo cliente (`nombre`, `cc`, `email`, `direccion`). Valida formato y unicidad de cédula y correo.
- `PUT /clientes/{id}`: Actualiza los datos de un cliente existente (`nombre`, `cc`, `email`, `direccion`).
- `DELETE /clientes/{id}`: Elimina un cliente por su identificador.

Servicio de Facturación (puerto 8084)

- `POST /facturas`: Endpoint único. Recibe `cliente_id` y lista de `items` (`producto_id` y `cantidad`). Consume los servicios de Auth, Clientes y Productos, calcula subtotales y total, asigna `factura_id` autoincremental y `fecha` actual, guarda en `data/facturas.json` y retorna el JSON de la factura.

2. Flujos del Sistema

Flujo 1: Autenticación de Usuario (Registro y Login)

1. El usuario envía `POST /auth/register` con su nombre, correo y contraseña. El servicio valida formato, comprueba que el correo no exista, cifra la contraseña con `bcrypt` y guarda el registro en `data/usuarios.json`.
2. El usuario envía `POST /auth/login` con correo y contraseña. El servicio busca el usuario en `data/usuarios.json`, verifica el hash con `bcrypt.CompareHashAndPassword` y, si es correcto, genera y responde con el token JWT (`HS256`) y los datos del usuario.

3. El frontend almacena el token JWT en `localStorage` para incluirlo en la cabecera `Authorization: Bearer <token>` de las solicitudes subsiguientes.

Flujo 2: Operaciones CRUD (Productos y Clientes)

1. **Productos:** Las operaciones `POST`, `GET`, `PUT`, `DELETE` sobre `/productos` son atendidas directamente por el microservicio en el puerto `8082` previa validación del token JWT mediante middleware. Las modificaciones actualizan el archivo `data/productos.json`.
2. **Clientes:** Las operaciones `POST`, `GET`, `PUT`, `DELETE` sobre `/clientes` son atendidas por el microservicio en el puerto `8083` previa validación del token JWT mediante middleware. El frontend permite registrar clientes mediante el formulario web. Las modificaciones actualizan `data/clientes.json`.

Flujo 3: Generación de Factura

1. **Petición del Cliente:** El usuario selecciona en el frontend un cliente destinatario y agrega productos con sus cantidades deseadas al carrito. Hace clic en "Generar Factura", enviando `POST /facturas` al servicio de Facturación con el token JWT en la cabecera y el cuerpo con `cliente_id` e `items`.
2. **Consumo de Auth:** Facturación realiza una petición `GET /auth/me` con el token JWT hacia el servicio de Autenticación (`:8081`) para verificar validez y obtener el usuario autenticado (`id`, `nombre`, `rol`).
3. **Consumo de Clientes:** Facturación realiza `GET /clientes/{cliente_id}` hacia el servicio de Clientes (`:8083`) para obtener los datos del cliente (`id`, `nombre`, `email`).
4. **Consumo de Productos:** Por cada item en el pedido, Facturación realiza `GET /productos/{producto_id}` hacia el servicio de Productos (`:8082`) para obtener el nombre oficial y precio unitario vigente.
5. **Cálculo y Consolidación:** Facturación calcula los subtotales (`cantidad * precio_unitario`) y la sumatoria total, asigna el identificador secuencial `factura_id` y la `fecha` en formato `YYYY-MM-DD`.
6. **Persistencia y Respuesta:** La factura completa se almacena de forma concurrente y atómica en `data/facturas.json` y se retorna con código HTTP `201 Created` al frontend en formato JSON.
7. **Visualización e Impresión:** El frontend renderiza la factura con sus datos formales y permite al usuario imprimirla o guardarla en PDF con las reglas `@media print` de `window.print()`.

(c) Ejemplo de Factura con Esquema Exacto

Esquema único: `{ "factura_id", "cliente": {"id","nombre","email"}, "usuario": {"id","nombre","rol"}, "productos": [{"id","nombre","cantidad","precio_unitario","subtotal"}], "total", "fecha": "YYYY-MM-DD" }`.


```
{
  "factura_id": 5001,
  "cliente": {
    "id": 201,
    "nombre": "Carlos Pérez",
    "email": "carlos@example.com"
  },
  "usuario": {
    "id": 1,
    "nombre": "Ricardo Gómez",
    "rol": "admin"
  },
  "productos": [
    {
      "id": 101,
      "nombre": "Laptop Lenovo ThinkPad",
      "cantidad": 1,
      "precio_unitario": 3500000.00,
      "subtotal": 3500000.00
    },
    {
      "id": 102,
      "nombre": "Mouse inalámbrico Logitech",
      "cantidad": 2,
      "precio_unitario": 12000.00,
      "subtotal": 24000.00
    }
  ],
  "total": 3524000.00,
  "fecha": "2026-10-18"
}
```